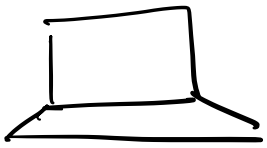
APIs

HTTP

Application Protocols

RPC
Remote Procedure call (Execute code not on single local m/c but on remote servers



Client

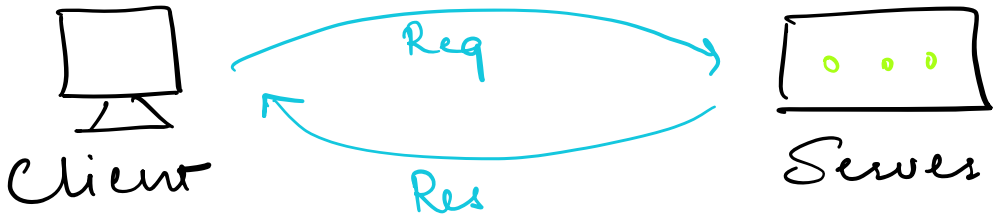
Client can be a computer or a server

Server

Computer in WH or DC

Note:- All Application layer protocols at core are RPCs. Everything in terms of Network calls fits in RPC
eg HTTP, Websocket

1) HTTP (Protocol of the Internet)
HTTP (Request Response Protocol) is built on TCP



Methods: GET, POST, PUT, DELETE

https://youtube.com → API Route
(" GET / POST → different End

PUT
DELETE] points

Idempotent GET :- To get or retrieve data (No Body)

POST :- To create data eg create an account (Body)

PUT :- To update data (Body)

Idempotent DELETE : To delete data (Body)

STATUS CODE :-

Informational responses (100-199)
Successful responses (200 - 299)
Redirection messages (300 - 399)
Client error responses (400 - 499)
Server error responses (500 - 599)

SSL/TLS

SSL (×) → Secure Socket Layer

TLS (✓) → Transport Layer Security

HTTPs (Hyper Text Transfer Protocol (secure)) SSL

Basically Browsers uses the SSL/TLS to make a secure connections to the website on the server also the site should configured with SSL/TLS certificate

Avoids :- Man in the Middle Attack

2) Web Sockets (Real Time Chat App)

Diff App layer Protocol

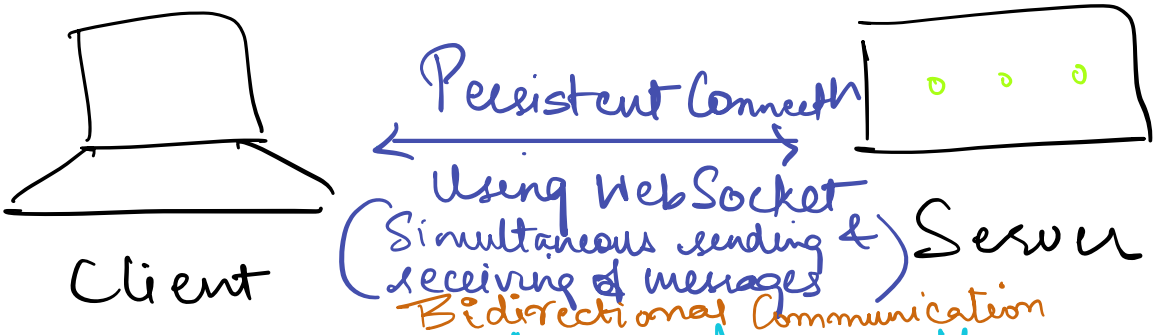
HTTP port 80

Built on TCP

- Web Socket:- To fill in the drawbacks of HTTP
- FTP - File Transfer Protocol
- SMTP - Secure Mail Transfer Protocol
- SSH - Connect to Remote PC to Run CLI

UDP

- WebRTC:- Video & Audio Stream



Say we want to get the latest chat from chat App like twitch but for that using HTTP we will have to send req to the server every second which can turn out to be expensive. For that WebSockets can help.

(Polling)

Note:- HTTP can help establishing web socket connection using Web Socket Handshake.

Web Sockets → HTTP2

Adv Web Sockets

A) Need messages in Real Time
B) Order of the messages also matter
C) Keep the communication going in both direction and no need of polling

Disadv Web Sockets

A) If the WebSocket Connection breaks the server might continue to send data

3) API Paradigms

API - Application Programming Interface (external service)

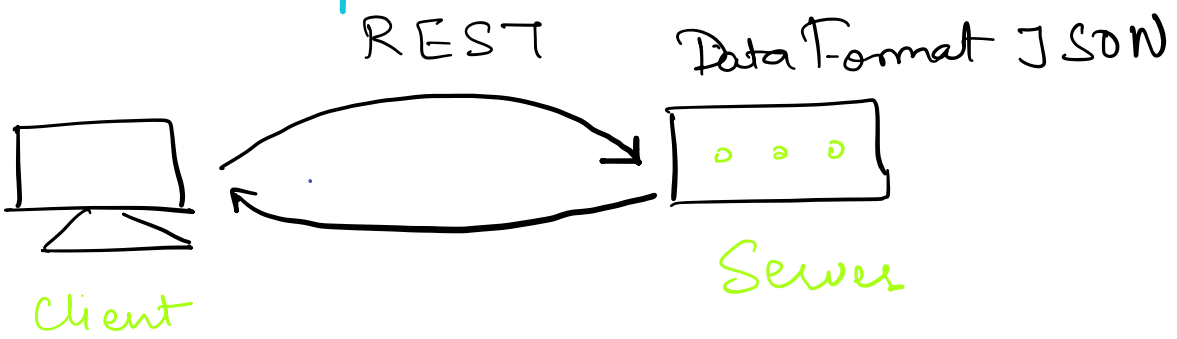
API - Is interface that serves user requests

API Paradigms

REST GraphQL gRPC

* REST (Representation State Transfer)

→ REST APIs are built on top of HTTP basically restriction applied on HTTP



→ REST is stateless meaning there is no data/record being stored in the server. It can be stored in an external storage

Pagination → We send the statefull information in the request as a query parameter

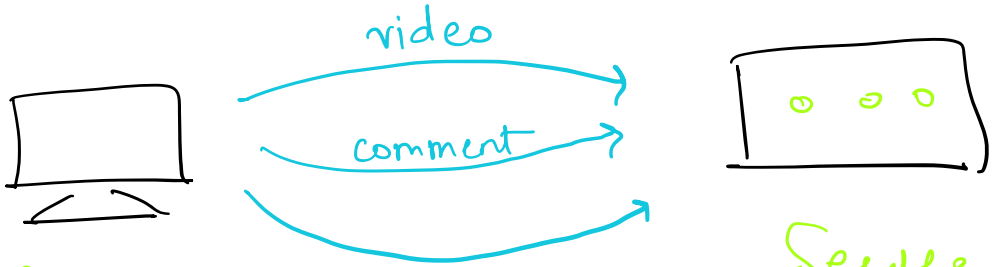
eg:- GET (Since does not have a body)
https://youtube.com/videos/:id → URL parameter
https://youtube.com/videos?offset=10
&limit=100
Query parameter

[Every EndPoint needs to be related to some resource]

→ Benefit is REST allows Horizontal scaling since the server do not have to store the state info each time

GraphQL API

→ GraphQL is built on HTTP POST - No caching
Request it uses the Body to include a Query to get exact resources from the Server



uses (Overfetching/ Underfetching)

Eg:- We want 5 comments from the user than we do not necessarily want all the User details just profile pic, name & comments
But REST may end up overfetching
however that can be avoided by having a custom logic on the server side

With GraphQL for every single resources we can specify the fields we want (Pagination) so the server can send only those fields to Client

Adv:- Save performance cost [less data sent so app will be faster]

GraphQL also solves under fetching problem

GraphQL
↓
Single Endpoint
- opt → Query
- opt → Mutation

* gRPC

→ gRPC is built on HTTP 2 (HTTP2 makes WebSocket absolute)
if using from browser it needs some proxy → difficult

→ Used for Server to Server communication

→ Faster than REST API

→ Sends information in Protocol Buffers – Schema Objects

→ Data Objects are serialized into Binary Objects – Smaller amount of data

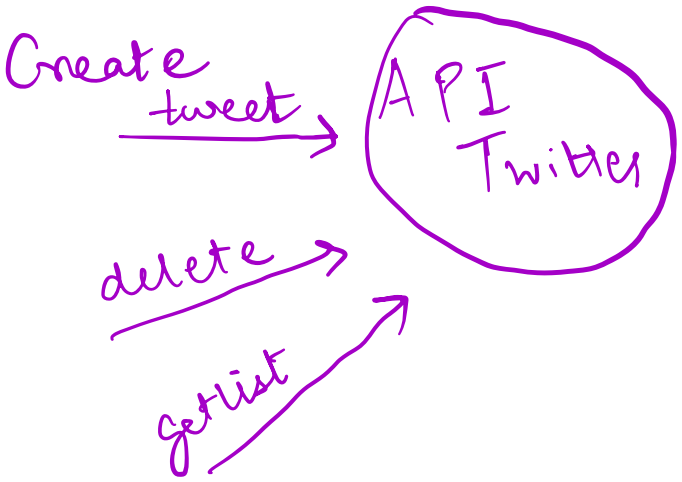
→ Allows Streaming & Bidirectional communication

→ Does not use HTTP Status code but error msg

Dis Adv :- New, less Standardized, Difficult Protocol

4) API Design

It is only about the surface area of the API (how to interact, CRUD operations)



⁂ We need to ensure our API is backward compatible

Meaning we already have designed an API now we want an additional mandatory parameter this will create an issue for current users using old API design.

API endpoint { Create Tweet(userId, content)

https://api.twitter.com/v1.0/tweet - POST

↑ Versioning

To get an individual tweet { https://api.twitter.com/v1.0/tweet/:id - GET

Pagination → EP parameters → limit, EP parameters → offset

Pagination

To deal with large amount of data or entries so as it does not crash the end users browsers.

https://api.twitter.com/v1.0/users/:id/tweets?limit=10&offset=0 - GET

uri parameter

Good API design — 1) Endpoint clearly defined
2) Backward Compatible
3) Pagination